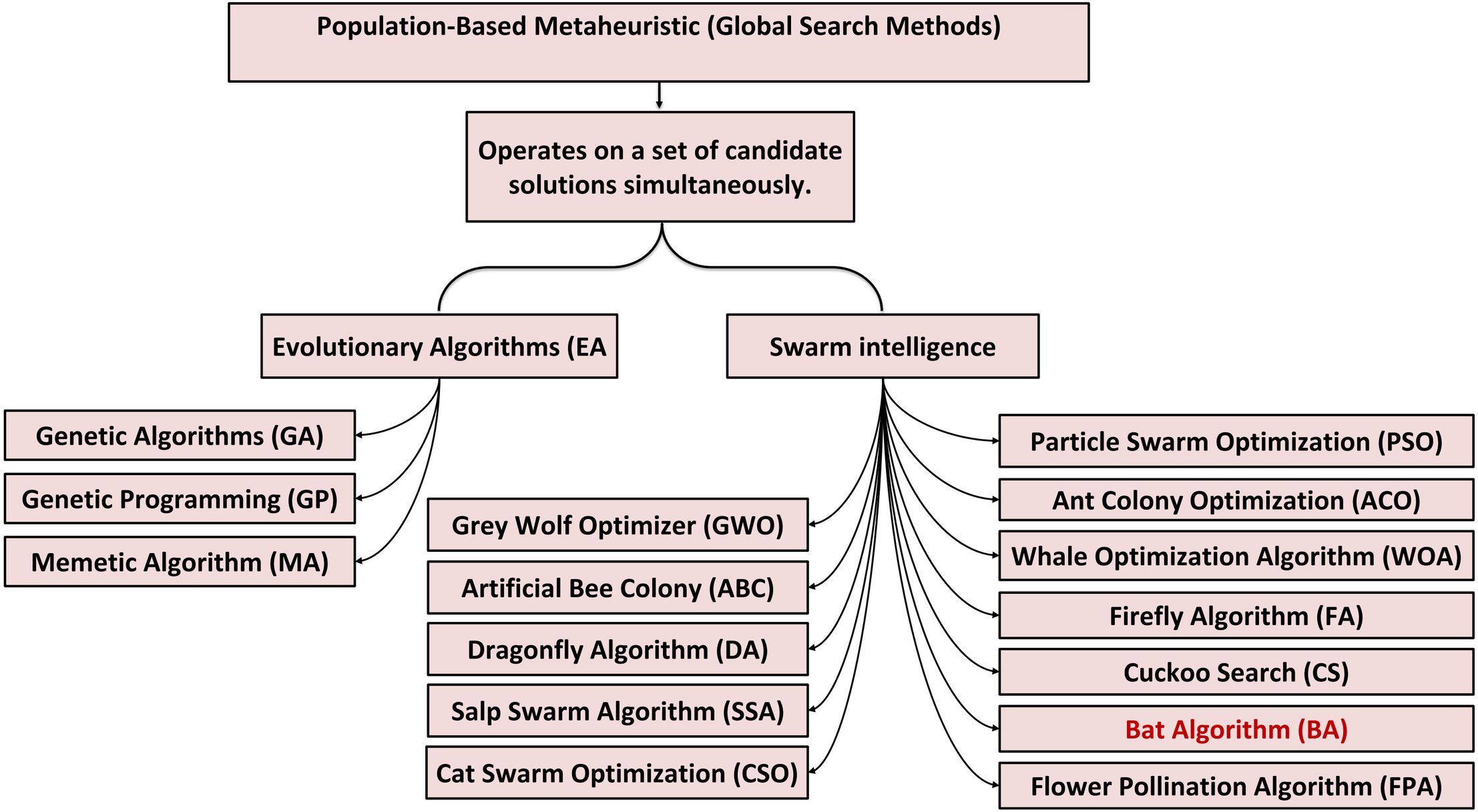
Nature Inspired Computation
DSAI 403

Assoc. Prof. **Mohamed Maher Ata**
Zewail city of science, technology, and innovation
momaher@zewailcity.edu.eg

Population-Based Search (Global Search)

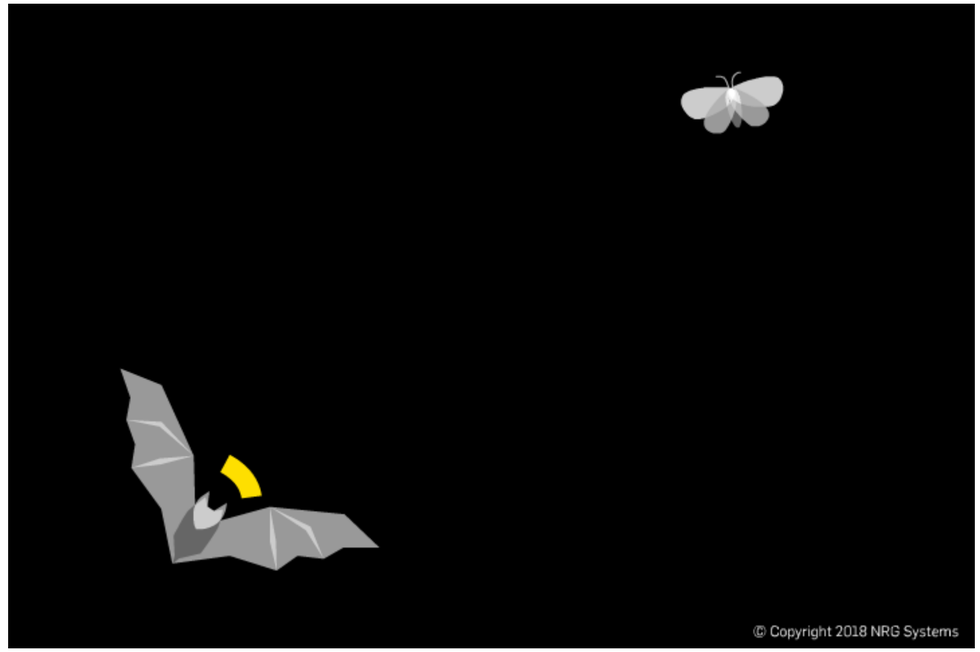


Bat Calls: Frequency Range and Function

- The frequency of bat sound depends on the species, because different bats use **echolocation** differently.
- Frequency range: Most bats emit ultrasonic sounds between 20 kHz and 200 kHz. Typical values:
  1) Common insect-eating bats: 40–80 kHz
  2) Horseshoe bats: up to 200 kHz
  3) Fruit bats: lower frequencies, sometimes near 20 kHz

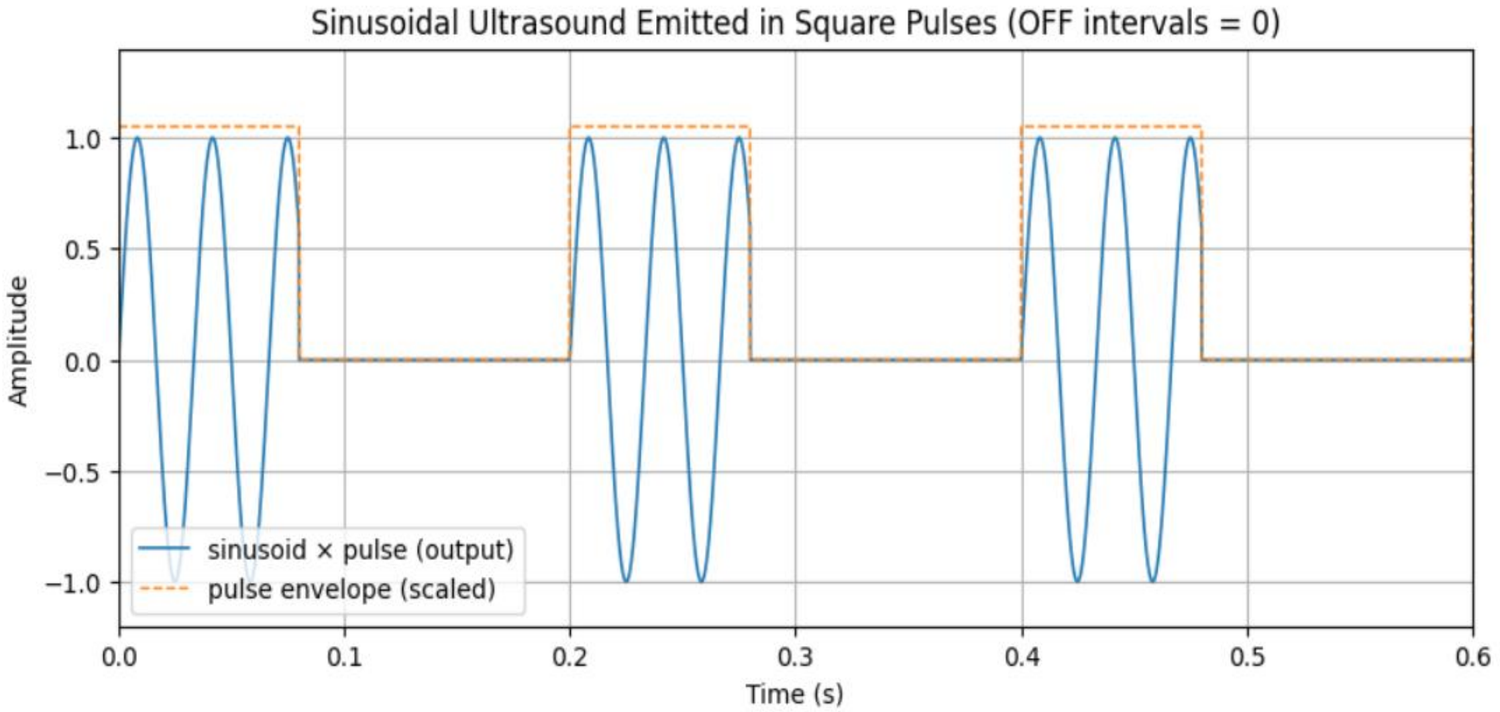
Notes:

- Humans generally cannot hear these sounds because they are above the human hearing range (20 Hz–20 kHz).
- Frequency affects the resolution of echolocation: higher frequencies give better precision but shorter range.



Echolocation

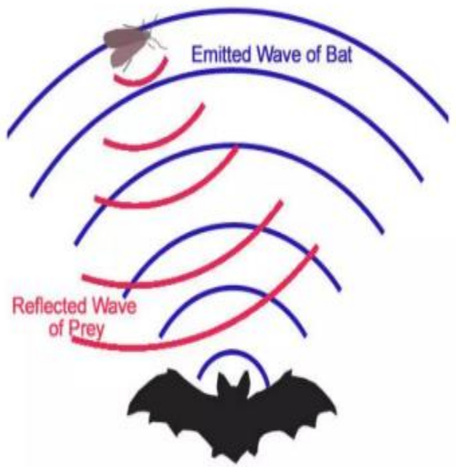
- The **Bat algorithm (BA)** is inspired by the echolocation behavior of microbats, with varying pulse rates of emission and loudness (improvising process of composing a piece of music).
- The bat emits sound = sinusoidal wave (ultrasound). But it does not make sound all the time. It sends that sound in short bursts (pulses).
  1) When sound is ON → it's like the square wave = 1.
  2) When sound is OFF → it's like the square wave = 0.
- Sinusoidal wave × Square pulse → Intermittent sinusoidal pulses

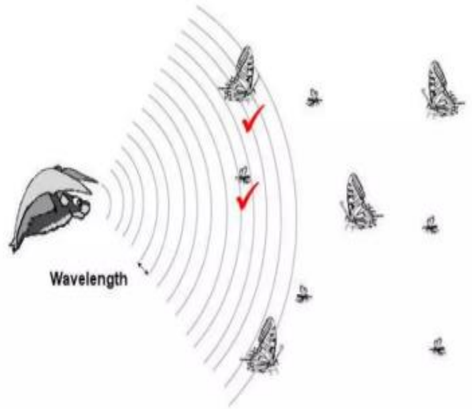


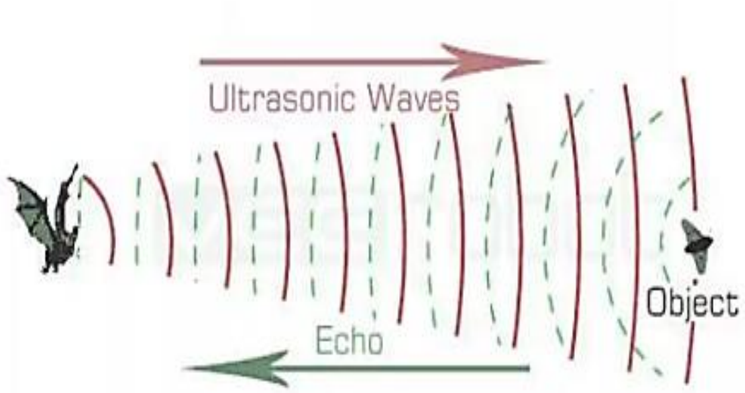
How Real Bats Use Echolocation

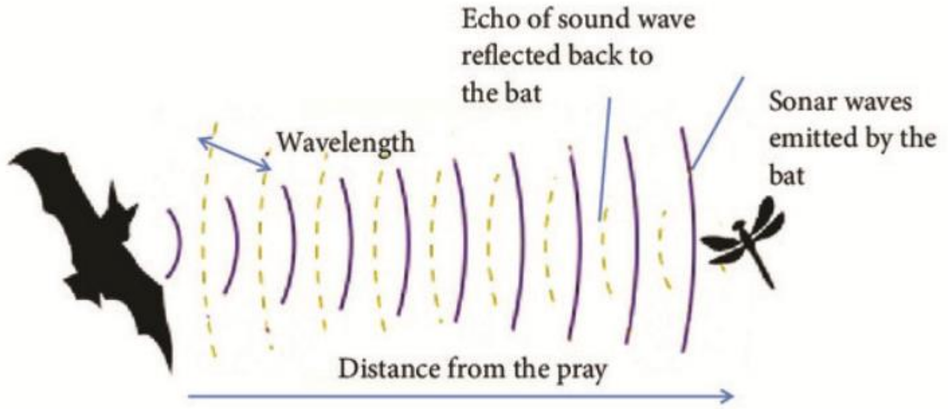
- Bats navigate and hunt in the dark using ultrasonic sound pulses.
- Each bat emits sound waves and listens to the echo reflected from objects or prey.
- From the echo, a bat can estimate:

1) Distance to prey or obstacle.
2) Direction of movement.
3) Speed of the target.



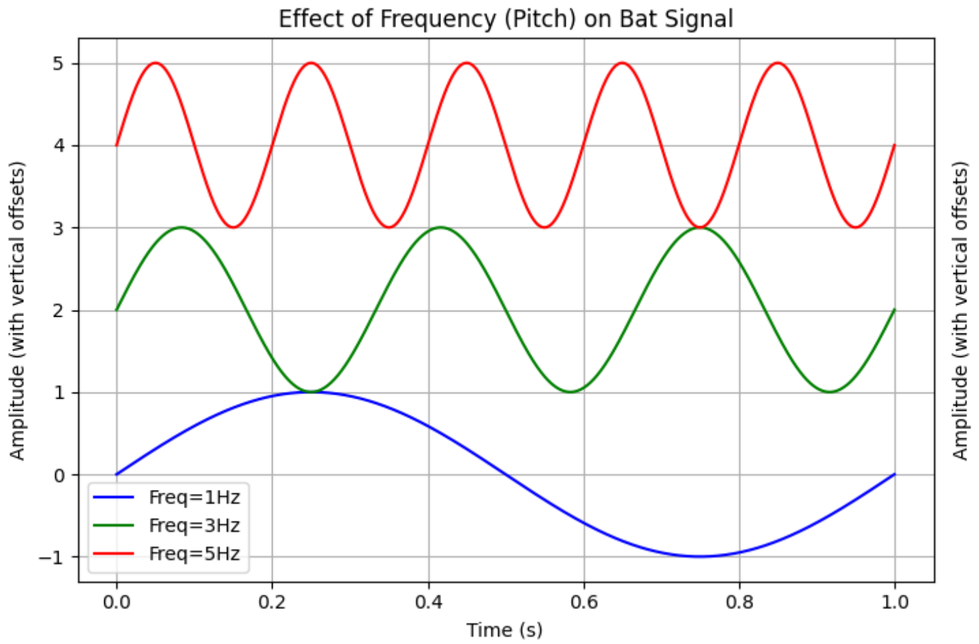






Basic parameters of BA

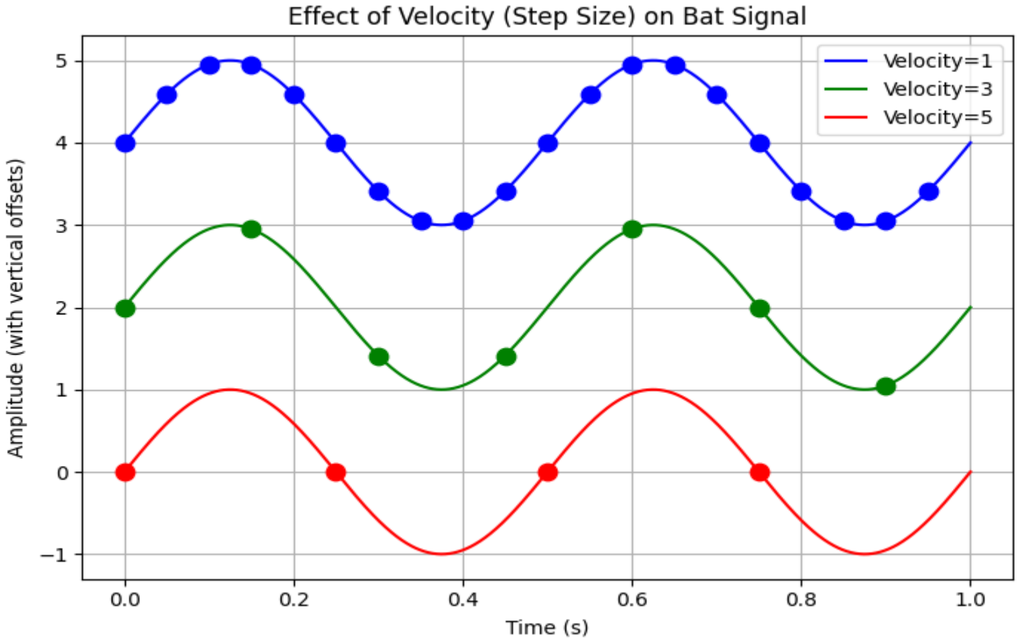
A) Frequency: signal rate



1) Low frequency: The sound wave travels farther → the bat can detect objects at **longer distances**, but with less detail.

2) High frequency: The sound wave travels shorter distances → the bat detects objects **closer**, but with more detail (better resolution).

B) Velocity: step size

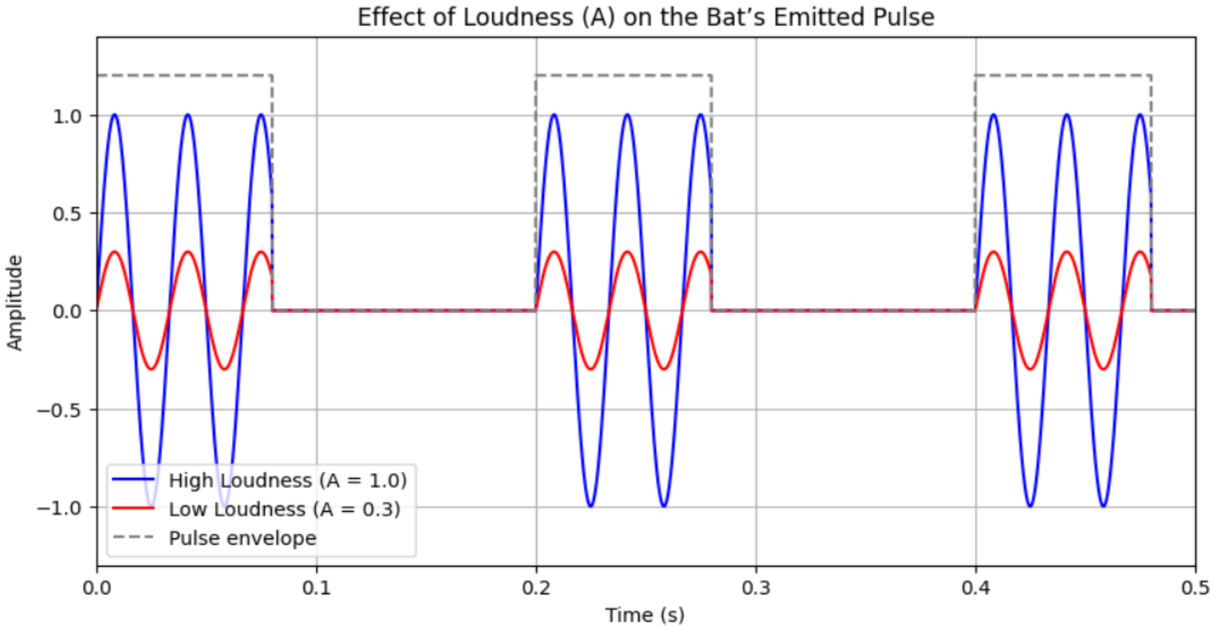


1) Low velocity: The bat takes small steps → focuses on exploitation → carefully searches around the current solution to improve it.

2) High velocity: The bat takes large steps → focuses on exploration → moves widely in the search space to find new promising areas.

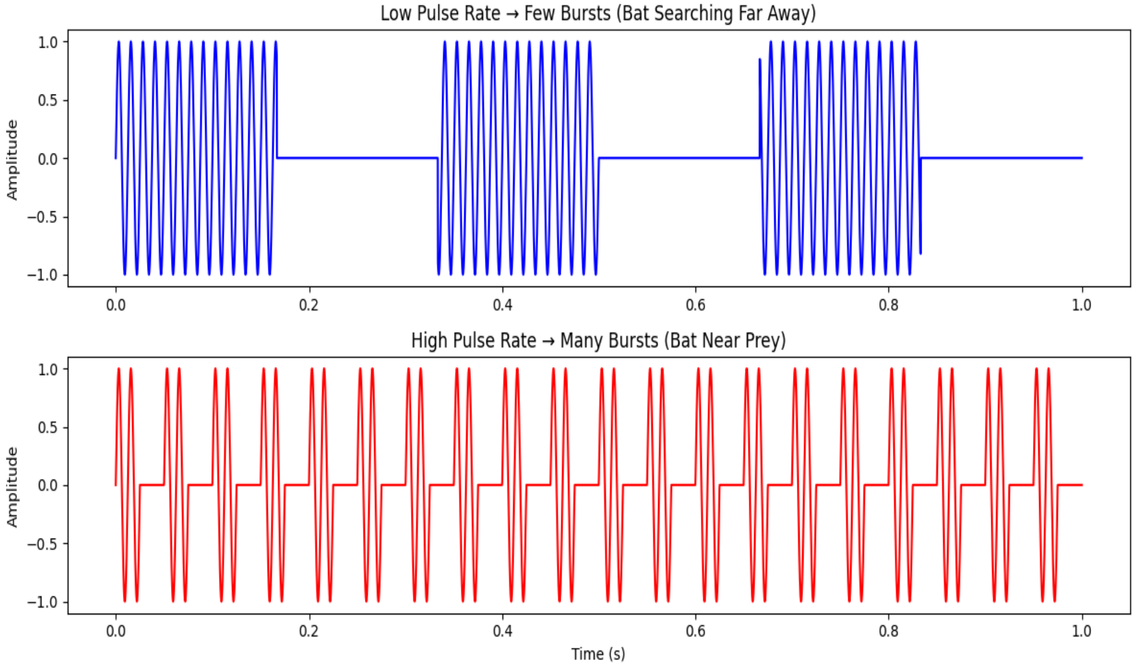
Basics of acoustic signal

C) Loudness



1) High loudness: Bat focuses on exploration → tries new solutions and searches widely.
2) Low loudness: Bat focuses on exploitation → refines current solutions carefully.

D) Pulse rate



3) Low pulse rate: Encourages exploration → searches globally and looks for new areas.
4) **High pulse rate**: Encourages exploitation → focuses locally around the current best solution.

BAT ALGORITHM

Why loudness falls while pulse rate rises

EXPLORATION





PREY

SEARCH A WIDE AREA

Strong calls • Fewer pulses • Broad exploration

LOUDNESS 85%

Call strength / amplitude

PULSE RATE 20%

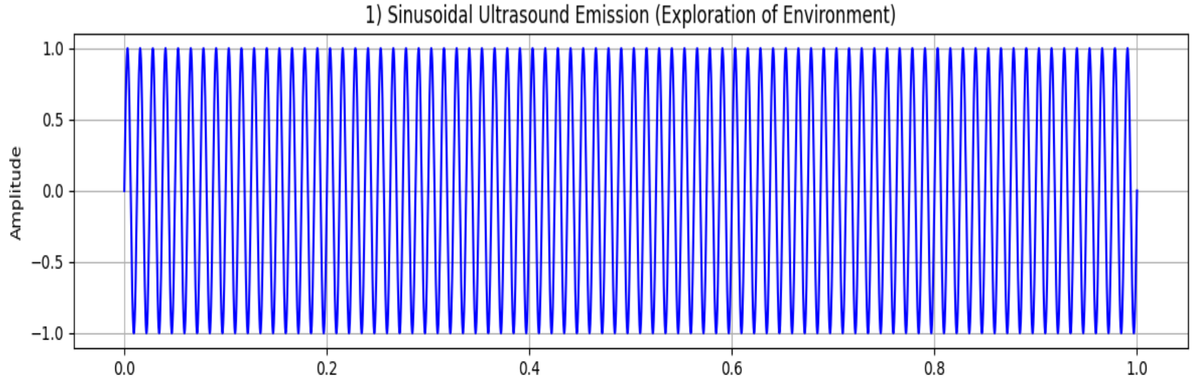
Pulses emitted per unit time

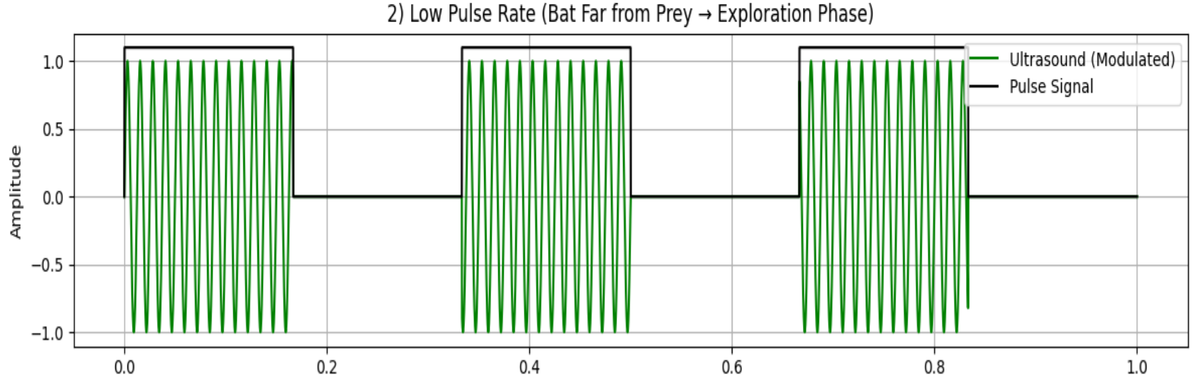
THE KEY TRADE-OFF

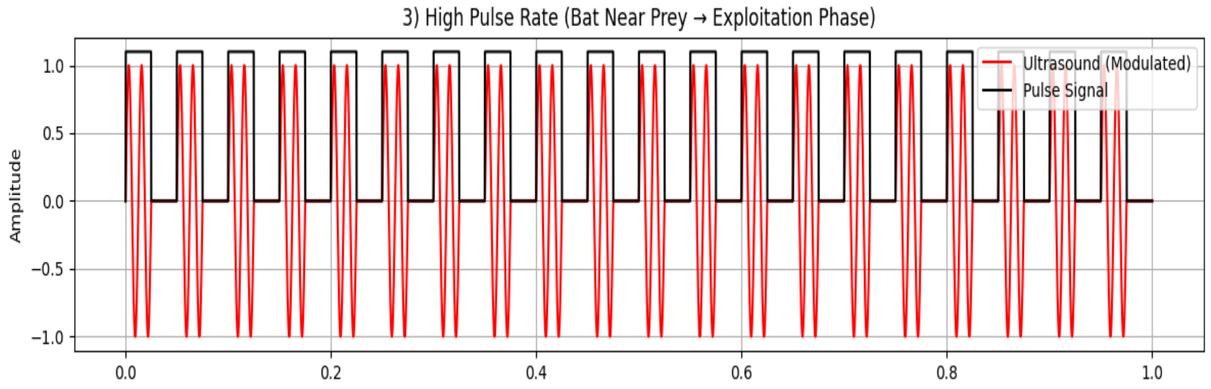
Explore: loud + infrequent

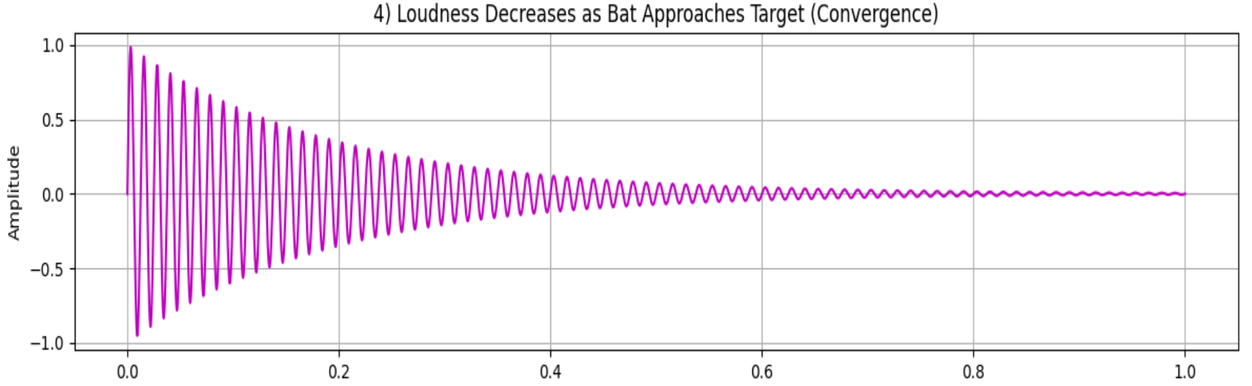
Exploit: quiet + frequent

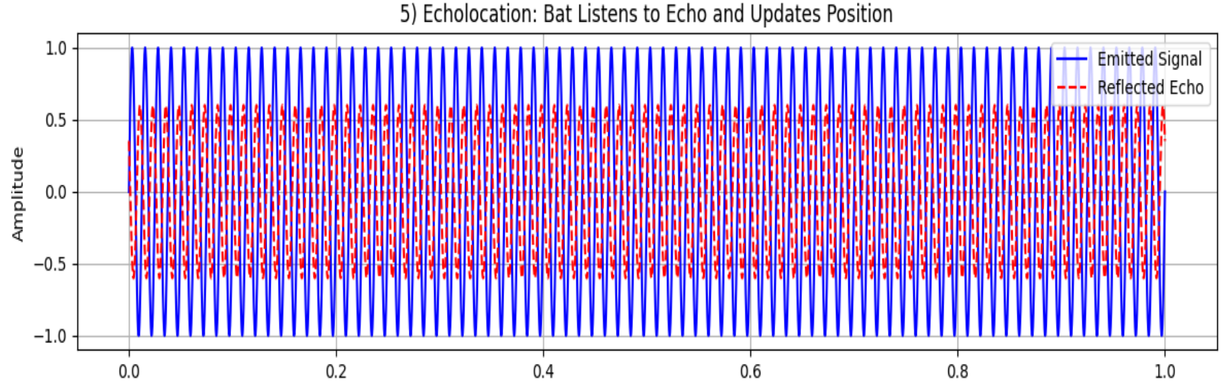
Bats Echolocation story

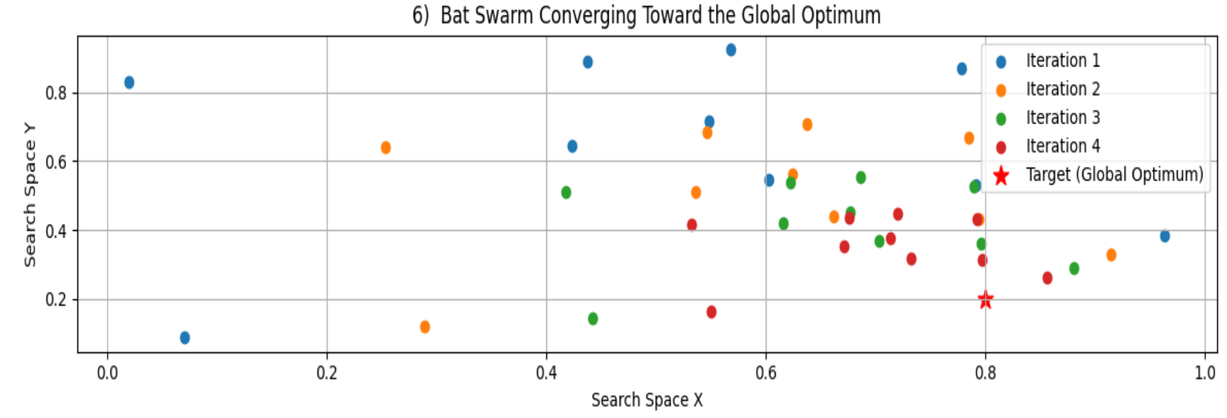












BA basic parameters

Parameter	In Real Bats (Biological Meaning)	In the Algorithm (Optimization Meaning)
Frequency	▪ Bats change the frequency of their sound waves to detect prey at different distances. ▪ Higher frequency → detects smaller objects. Lower frequency → scans a wider area.	▪ Controls how far a bat moves in the search space ▪ low frequency means long jumps (global search or exploration) ▪ High frequency means small, precise moves (local search or exploitation).
Velocity	▪ Represents how fast and in which direction a bat flies toward its prey based on sound feedback.	▪ Defines the **step size** and direction toward the best-known solution.
Loudness	▪ As bats approach their prey, they reduce the loudness of their emitted sounds to focus on fine-tuning detection.	▪ Controls acceptance of new solutions ▪ High loudness allows random exploration ▪ Low loudness forces the algorithm to focus near the best solution (exploitation)
Pulse Rate	▪ When a bat gets closer to the prey, it emits pulses more frequently to get detailed information.	▪ Controls how often local search occurs ▪ low pulse rate means global search ▪ high pulse rate means detailed local refinement.

Optimization through Echolocation Behavior

- Each bat = one candidate solution in the search space.
- The goal: find the position (solution) with the best fitness (lowest or highest value).
- Bats “fly” toward the best solution using:

  1) Frequency (f): how quickly a bat changes direction: $\lambda(wave\ length) = \frac{C}{f}$

  2)Velocity (v): step size toward the prey (solution).

  3)Loudness (A) (Amplitude of signal): how likely the bat accepts new solutions (decreases near prey).

  4) Pulse rate (r): how often a bat performs local search (increases near prey).

Note that:

- At early iterations → bats fly randomly (exploration).
- Later → they reduce loudness and fine-tune near prey (exploitation).

Mathematical model

Each bat i at iteration t has:

- Position → x_i^t
- Velocity → v_i^t
- Frequency → f_i
- Loudness → A_i^t
- Pulse rate → r_i^t

Step 1: Frequency Update:

$$f_i = f_{\min} + (f_{\max} - f_{\min}) \cdot \text{rand}()$$

Where:

- rand()∈[0,1]
- f_i controls how far the bat moves in the search space.
- Low frequency → detect far objects (global search)
- High frequency → detect nearby objects (local search)

Step 2: Velocity and Position Update

$$v_i^t = v_i^{t-1} + \left(x_i^{t-1} - x_*\right) \cdot f_i$$

$$x_i^t = x_i^{t-1} + v_i^t$$

Where:

- x_*: current global best position
- Bats move toward the global best while maintaining diversity through frequency control.

Step 3: Local Random Walk

If a bat is near the prey (solution):

$$x_{\text{new}} = x_* + \epsilon A_i^t$$

Where:

- $\epsilon \in [-1,1]$ is a random small number
- A_i^t: Average loudness of all bats

Step 4: Loudness and Pulse Rate Update

$$A_i^{t+1} = \alpha A_i^t$$

$$r_i^{t+1} = r_i^0[1 - e^{-\gamma t}]$$

Where:

- $\alpha, \gamma \in (0,1)$ are constants controlling convergence.
- $A_i \downarrow \Rightarrow$ bats become quieter → exploitation
- $r_i \uparrow \Rightarrow$ more local search near best → refinement

Why step 4

In nature, when a bat approaches its prey:

- It **reduces its loudness** (to stay quiet),
- And **increases its pulse rate** (more frequent sonar pings).
- This behavior means the bat is **confident** it's near the target.

Example: we need to minimize $f(x) = x^2$.

- Algorithm parameters: $f_{min} = 0, f_{max} = 2,$ loudness reduction factor $(\alpha = 0.9)$, Pulse-rate growth coefficient $(\gamma = 0.9)$, initial loudness for all bats $(A_i^0 = 1)$, initial pulse rate for all bats $(r_i^0 = 0)$, max pulse rate for all bats $(r_i^{max} = 0.1)$

- Consider the following initial data:

Bat	$\mathbf{x_i^t}$ (initial positions)	$\mathbf{v_i^t}$ (Initial velocities)	Random value for frequencies	$\mathbf{rand}_{\text{pulse } rate}$	$\mathbf{rand}_{accept}$	$\boldsymbol{\varepsilon}$ (random number for local walk)
B1	2.0	0.0	0.25	0.20	0.4	-0.5
B2	-1.0	0.0	0.75	0.05	0.6	0.2
B3	0.5	0.0	0.1	0.30	0.8	0.2

Step (1):

Bat	x_i^t	$f(x_i^t) = x^2$
B1	2.0	$2^2 = 4$
B2	-1.0	$(-1)^2 = 1$
B3	0.5	$(0.5)^2 = 0.25$

→ Current global best: **Bat: B3** → ($x_* = 0.5$)

Step 2: Compute frequency:

$$f_i = f_{\min} + (f_{\max} - f_{\min}) \times rand$$
$$f_1 = 0 + (2 - 0) \times 0.25 = 0.5$$
$$f_2 = 0 + (2 - 0) \times 0.75 = 1.5$$
$$f_3 = 0 + (2 - 0) \times 0.1 = 0.2$$

Step 3 - Velocity update

$$v_i^t = v_i^{t-1} + \left(x_i^{t-1} - x_*\right) \cdot f_i$$

- **Bat 1**: $(x_1 - x_*) = 2 - 0.5 = 1.5$
  $v_1 = 0 + 1.5 \times 0.5 = \mathbf{0.75}$
- **Bat 2**: $(x_2 - x_*) = -1 - 0.5 = -1.5$
  $v_2 = 0 - 1.5 \times 1.5 = \mathbf{-2.25}$
- **Bat 3**: $(x_3 - x_*) = 0.5 - 0.5 = 0$
  $v_3 = 0 + 0 \times 0.2 = \mathbf{0}$

Step 4: Position update (global move)

$$x_i^{t+1} = x_i^t + v_i^{t+1}$$

- **Bat 1**: $x_1 = 2 + 0.75 = 2.75$ → **Fitness:** $f(2.75) = 7.5625$ (initial was $2^2 = 4$) → (worse)
- **Bat 2**: $x_2 = -1 - 2.25 = -3.25$ →**Fitness:** $f(-3.25) = 10.5625$ (initial was $-1^2 = 1$) → (worse)
- **Bat 3**: $x_3 = 0.5 + 0 = 0.5$ → **Fitness:** $f(0.5) = 0.25$ (initial was $0.5^2 = 0.25$) → **(unchanged)**

Step 5: Local random walk: $(\boldsymbol{rand_{pulse} > r_i})$

Bat	$\mathbf{rand_{pulse}}$	$\mathbf{r_i}$ (pulse rate)	Local move?
B1	0.20	0	True
B2	0.05	0	True
B3	0.30	0	True

Step 6: Generate new candidate solutions around the current best solution to improve exploitation: this step is responsible for determining which bats should intensify their search around a promising solution.

$$x_{\text{new}} = x_* + \epsilon\overline{A^t}\text{, where } \epsilon \in [-1,1] \text{ is a random small number}$$

Now the new candidate positions are:

Where: $\overline{A^t}$ is the average loudness of all bats $(\frac{1+1+1}{3}) = 1$

- **Bat 1: choose** $\epsilon = -0.5$ → $x_{local} = 0.5 + (-0.5) \times 1 = 0$ → Fitness: $f(0) = 0$
- **Bat 2: choose** $\epsilon = 0.2$ → $x_{local} = 0.5 + (0.2) \times 1 = 0.7$ → Fitness: $f(0.7) = 0.49$
- **Bat 3: choose** $\epsilon = 0.2$ → $x_{local} = 0.5 + (0.2) \times 1 = 0.7$ → Fitness: $f(0.7) = 0.49$

Step 7: Candidate Acceptance (Both conditions must hold):

$rand_{accept} < A_i^t$ && $f(x_{candidate}) < f(x_*)$

Bat	Old x	New x	New f	rand acc.	rand < A?	f < 0.25?	Decision	Final x	Final f
B1	2.0	0	0	0.4	Yes	Yes	Accept	0	0
B2	−1.0	0.7	0.49	0.6	Yes	No	Reject	−1.0	1
B3	0.5	0.7	0.49	0.8	Yes	No	Reject	0.5	0.25

Step 8 : Loudness Update (this step is responsible for reducing loudness after an accepted move to refine the search).

Bat	Accepted in Step 7?	Old A_i	$A_i^{t+1} = \alpha A_i^t$	New A_i
B1	Yes	1.0	0.9 × 1.0 = 0.9	0.9
B2	No	1.0	No update	1.0
B3	No	1.0	No update	1.0

Step 9: Pulse-Rate Update (this step is responsible for adapting the probability of selecting local search for future iterations).

Bat	Accepted in Step 7?	Old r_i	$r_i^{t+1} = r_i^{\max}[1 - e^{-\gamma t}]$ **Where** $t = 1 \rightarrow$ current iteration number	New r_i
B1	Yes	0.1	0.1 × [1 − exp(−0.9)]	0.05934
B2	No	0.1	No update	0
B3	No	0.1	No update	0

Final Results — Next Iteration

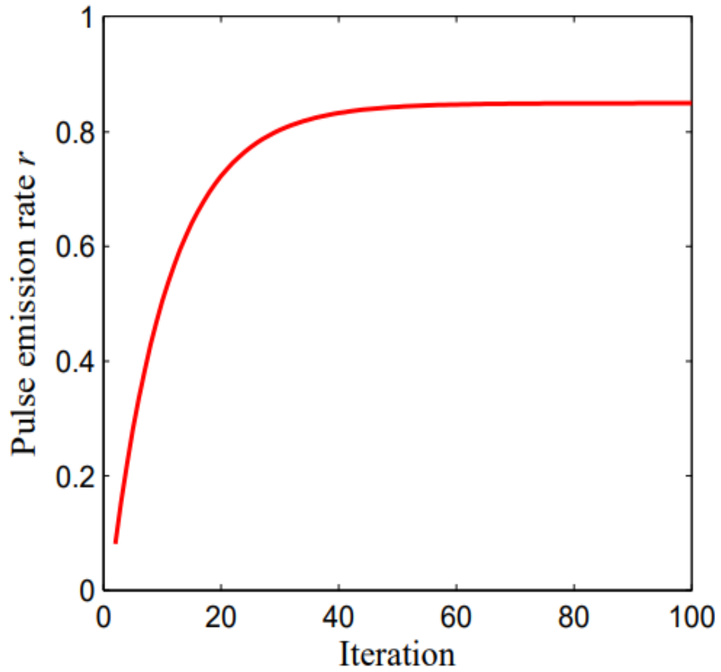
Parameter	B1	B2	B3
Accepted position	**0**	−1	0.5
Velocity	**0.75**	−2.25	0
Fitness	**0**	1	0.25
Loudness	**0.9**	1.0	1.0
Pulse rate	**0.05934**	0	0

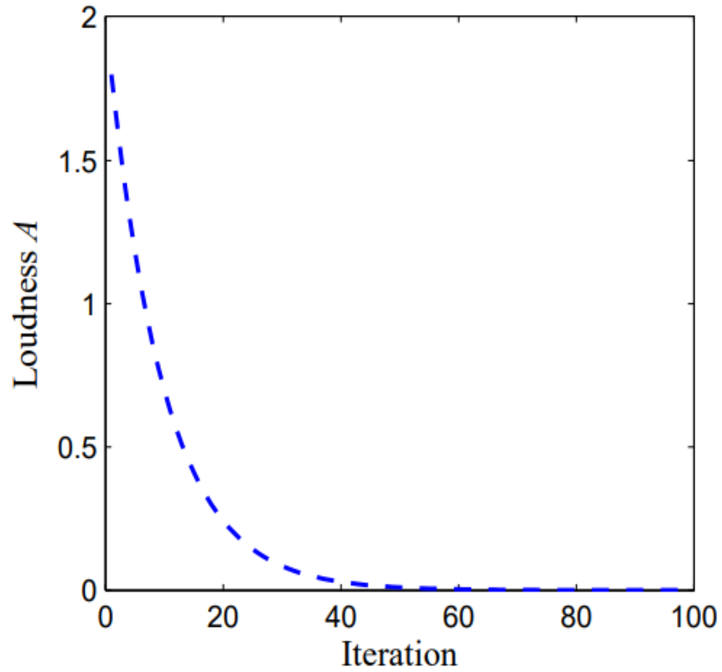
Important note :

In the Bat Algorithm, the **pulse rate starts at 0** and gradually increases as the search progresses and better solutions are accepted. We define the **Maximum Pulse Rate (0.1)** as the upper limit that the pulse rate approaches over time. This distinction is important because using an initial pulse rate of 0 directly in the original update equation would always produce 0 due to multiplication by zero, resulting in a trivial solution with no pulse-rate growth. By using a maximum pulse rate of 0.1, the pulse rate can increase from 0 to 0.05934 after the first accepted iteration and gradually approach 0.1 in subsequent accepted iterations. This behavior helps balance exploration and exploitation, as a lower pulse rate triggers local search more frequently, while a higher pulse rate reduces its frequency. In the original paper, Yang refers to this parameter as the *initial pulse rate*, but we use **Maximum Pulse Rate** to clarify its mathematical role and avoid confusion. This is a clarification of the original formulation rather than a new algorithm.

Exploration and Exploitation in BA

	Loudness (A_i)	Pulse Rate (r_i)	Frequency (f_i)	Behavior
Exploration	High	Low	Low–Medium	Bats move globally, search widely
Exploitation	Low	High	High	Bats refine around best area





1. Load and prepare dataset:

```
iris = load_iris()
X = iris.data       # features: sepal length, width, etc.
y = iris.target.reshape(-1, 1)

# Normalize features
scaler = MinMaxScaler()
X = scaler.fit_transform(X)

# One-hot encode labels
encoder = OneHotEncoder(sparse_output=False)
y = encoder.fit_transform(y)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

2. Define simple Neural Network

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def forward_pass(X, weights):
    # Hidden layer (2 neurons) + Output layer (3 classes)
    w1 = weights[:8].reshape(4, 2)      # 4 inputs → 2 hidden
    w2 = weights[8:].reshape(2, 3)      # 2 hidden → 3 outputs

    hidden = sigmoid(np.dot(X, w1))
    output = sigmoid(np.dot(hidden, w2))
    return output

def compute_loss(X, y, weights):
    output = forward_pass(X, weights)
    loss = np.mean((y - output) ** 2)   # Mean Squared Error
    return loss
```

3. Initialize Bat Algorithm

```
num_bats = 3
num_features = 14       # total weights in small NN
max_iter = 10

f_min, f_max = 0, 2
alpha = 0.9
gamma = 0.9

# Initialize
positions = np.random.uniform(-1, 1, (num_bats, num_features))  # weights
velocities = np.zeros_like(positions)
frequency = np.zeros(num_bats)
loudness = np.ones(num_bats)
pulse_rate = np.zeros(num_bats)
fitness = np.array([compute_loss(X_train, y_train, p) for p in positions])

best_idx = np.argmin(fitness)
best_position = positions[best_idx].copy()
best_score = fitness[best_idx]

print("Initial best loss:", best_score)
```

$4 \times 2 + 2 \times 3 = 14$
(assuming no bias)

4. Optimization Loop

```
for t in range(1, max_iter + 1):
    for i in range(num_bats):
        # --- Step 4.1: Update frequency
        freq = f_min + (f_max - f_min) * np.random.rand()
        frequency[i] = freq
        # --- Step 4.2: Update velocity and position
        velocities[i] = velocities[i] + (positions[i] - best_position) * freq
        new_pos = positions[i] + velocities[i]
        # --- Step 4.3: Local random walk (exploration)
        if np.random.rand() > pulse_rate[i]:
            new_pos = best_position + 0.001 * np.random.randn(num_features)
        # --- Step 4.4: Evaluate new solution
        new_fitness = compute_loss(X_train, y_train, new_pos)
        # --- Step 4.5: Acceptance criteria
        if (new_fitness < fitness[i]) and (np.random.rand() < loudness[i]):
            positions[i] = new_pos
            fitness[i] = new_fitness
            loudness[i] *= alpha
            pulse_rate[i] = 0.1 * (1 - np.exp(-gamma * t))
        # --- Step 4.6: Update global best
        if new_fitness < best_score:
            best_score = new_fitness
            best_position = new_pos.copy()
    print(f"Iteration {t}: Best Loss = {best_score:.6f}")
print("\nOptimization completed!")
print("Best Loss =", best_score)
```

5. Evaluate on test data

```
y_pred = forward_pass(X_test, best_position)
y_pred_labels = np.argmax(y_pred, axis=1)
y_true = np.argmax(y_test, axis=1)

accuracy = np.mean(y_pred_labels == y_true)
print(f"Test Accuracy: {accuracy * 100:.2f}%")
```

```
Initial best loss: 0.23125929482862956
Iteration 1: Best Loss = 0.231211
Iteration 2: Best Loss = 0.231211
Iteration 3: Best Loss = 0.231206
Iteration 4: Best Loss = 0.231186
Iteration 5: Best Loss = 0.231186
Iteration 6: Best Loss = 0.231176
Iteration 7: Best Loss = 0.231148
Iteration 8: Best Loss = 0.231143
Iteration 9: Best Loss = 0.231115
Iteration 10: Best Loss = 0.231107

Optimization completed!
Best Loss = 0.2311072979586676
Test Accuracy: 30.00%
```

Note that:

- **Bat Algorithm (BA)** is indeed *optimizing* (loss is decreasing each iteration, but the **accuracy is low (30%)**, meaning: BA did not find very good weights within only 10 iterations and a small population (10 bats).
- It needs **more bats and iterations** to explore the solution space better.

Increase number of bats (in order to increase exploration), and increase number of iterations

```
num_bats = 100
num_features = 14       # total weights in small NN
max_iter = 50

f_min, f_max = 0, 2
alpha = 0.9
gamma = 0.9
```

New results:

- Best Loss = 0.1598107290989422
- Test Accuracy: 70.00%

```
Initial best loss: 0.22359363188877682
Iteration 1: Best Loss = 0.222692
Iteration 2: Best Loss = 0.221766
Iteration 3: Best Loss = 0.221089
Iteration 4: Best Loss = 0.220518
Iteration 5: Best Loss = 0.220050
Iteration 6: Best Loss = 0.219304
Iteration 7: Best Loss = 0.218915
Iteration 8: Best Loss = 0.218446
Iteration 9: Best Loss = 0.217810
Iteration 10: Best Loss = 0.217310
Iteration 11: Best Loss = 0.216819
Iteration 12: Best Loss = 0.216327
Iteration 13: Best Loss = 0.215847
Iteration 14: Best Loss = 0.215308
Iteration 15: Best Loss = 0.214790
Iteration 16: Best Loss = 0.214217
Iteration 17: Best Loss = 0.213660
Iteration 18: Best Loss = 0.213196
Iteration 19: Best Loss = 0.212713
Iteration 20: Best Loss = 0.212130
Iteration 21: Best Loss = 0.211561
Iteration 22: Best Loss = 0.211148
Iteration 23: Best Loss = 0.210570
Iteration 24: Best Loss = 0.210070
Iteration 25: Best Loss = 0.209588
Iteration 26: Best Loss = 0.209197
Iteration 27: Best Loss = 0.208655
Iteration 28: Best Loss = 0.208027
Iteration 29: Best Loss = 0.207526
Iteration 30: Best Loss = 0.207005
Iteration 31: Best Loss = 0.206404
Iteration 32: Best Loss = 0.205933
Iteration 33: Best Loss = 0.205382
Iteration 34: Best Loss = 0.204756
Iteration 35: Best Loss = 0.204131
Iteration 36: Best Loss = 0.203603
Iteration 37: Best Loss = 0.203215
Iteration 38: Best Loss = 0.202826
Iteration 39: Best Loss = 0.202285
Iteration 40: Best Loss = 0.201650
Iteration 41: Best Loss = 0.201102
Iteration 42: Best Loss = 0.200560
Iteration 43: Best Loss = 0.196264
Iteration 44: Best Loss = 0.185021
Iteration 45: Best Loss = 0.183641
Iteration 46: Best Loss = 0.182660
Iteration 47: Best Loss = 0.181242
Iteration 48: Best Loss = 0.169693
Iteration 49: Best Loss = 0.167722
Iteration 50: Best Loss = 0.159811

Optimization completed!
Best Loss = 0.1598107290989422
Test Accuracy: 70.00%
```

Applications of BA

1. **Neural network training**: optimizes network weights without using gradient descent
2. **Hyperparameter optimization**: tunes parameters such as learning rate, number of layers, or neurons
3. **Feature selection**: selects the most relevant input features to improve model accuracy
4. **Clustering**: finds optimal cluster centers for unsupervised learning
5. **Reinforcement learning and robotics**: optimizes control policies, robot paths, or movement parameters
6. **Optimization in security and encryption**: used for key generation and image encryption parameter tuning